Parser and Dependency Graph Validation

Task F

William A. Davis
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
October 10, 2026

Question and Method

Dependency depth is one of the five scoring factors, so the parser has to reproduce each project's dependency structure faithfully. To test this, I compared what the sbomrisk parser computes from each SBOM against an independent source of truth: the package manager of that project's own ecosystem, run on the same project. The comparison is automated by scripts/validate_graph.py, and the reference outputs are saved in data/validation.

I ran every check on my own Windows computer (Python 3.13.9 and pytest 9.1.1, with Java 25 and the Maven wrapper shipped with WebGoat for the Maven check), using the copy of the project downloaded from GitHub. All 16 automated tests passed. The sbomrisk inspect command reproduced every component, dependency link, direct-dependency and depth count reported in the test project document.

Results

Project	Reference tool	Components matched	Structure matched	Result
Juice Shop (npm)	npm ls [1]	645 of 645 (100%)	Depth: 645 of 645 (100%)	Pass
PyGoat (Python)	pipdeptree [2]	33 of 33 (100%)	Dependency links: 25 of 25, none extra	Pass
Gitea (Go)	go list -m all [3]	231 of 231 in Go's build list, same versions	Not applicable (see below)	Pass
WebGoat (Maven)	mvn dependency:tree [4]	179 of 179 (100%), plus 11 explained below	Depth: 179 of 179 (100%)	Pass

Problems Found and Resolved

The validation surfaced four issues. Two were weaknesses in the validation script itself, one was a genuine error in the data produced by an SBOM generator, and one was a quirk of how Maven prints its dependency tree.

1. Duplicate npm installations

npm can install the same version of a package in more than one place. For example, debug 4.4.3 is installed both under sequelize and under sqlite3, at different depths. The first version of the check kept whichever copy it encountered last, so fourteen packages appeared to disagree on depth. Using the shallowest copy, which is how depth is defined, produced complete agreement. This finding also changed the scoring specification: duplicate installations are now merged into a single ranked entry at their shallowest depth.

2. Renamed packages

npm allows a package to be installed under an alias (string-width-cjs is an alias for string-width), and Go allows a module to be replaced by another (Gitea replaces github.com/nektos/act with gitea.com/gitea/act). The check now resolves both kinds of renaming.

3. Optional extras recorded as required dependencies

This was a real data error. The Python SBOM generator recorded ten dependency links that pip does not treat as required, because they come from optional "extras." Examples are cryptography → pytz, which comes from cryptography's test extra, and Django → argon2-cffi, which comes from an optional Django extra. These false links placed three packages at the wrong depth and hid the fact that cryptography is one of PyGoat's direct dependencies. The helper script now removes any link that pipdeptree does not confirm. After the correction PyGoat has 16 direct dependencies rather than 14, and its maximum depth is 3 rather than 4. Without validation, this error would have distorted the depth factor for every affected package.

4. Libraries hidden by Maven's scope filter

The WebGoat SBOM contains eleven runtime libraries that `mvn dependency:tree -Dscope=runtime` does not print, including spring-security-core, spring-security-web, byte-buddy, json-smart and commons-logging. Each of these is also used by one of WebGoat's test dependencies, and through that test dependency it sits closer to the top of the tree. For example, spring-security-test → spring-security-core is two levels deep, compared with three through the runtime path. Maven draws each library only once, at its nearest position, so it draws these under the test dependency. The runtime filter then removes the test dependency's branch, and the library disappears from the printout even though it is part of the running application. The SBOM is correct to include these libraries at their runtime depth.

I confirmed this by printing the full tree without the scope filter. Each library appears under a parent marked test while being marked compile or runtime itself:

```
+- org.springframework.security:spring-security-test:jar:7.1.1:test
|  +- org.springframework.security:spring-security-core:jar:7.1.1:compile
|  +- com.jayway.jsonpath:json-path:jar:2.10.0:test
|  +- net.minidev:json-smart:jar:2.6.0:compile
|  +- org.assertj:assertj-core:jar:3.27.7:test
|  |  \- net.bytebuddy:byte-buddy:jar:1.18.11:runtime
```

What This Validation Does Not Cover

Go's module system records which modules a build selected, but not a parent-and-child tree of what is linked, so for Gitea only module membership and versions could be checked, not depth. Vulnerability matching and risk scores do not exist yet; they will be produced in Tasks G and H and validated against an independent scanner as part of the evaluation (D3).

References

[1] npm, Inc., "npm ls," npm Docs. [Online]. Available: https://docs.npmjs.com/cli/commands/npm-ls

[2] tox-dev, "pipdeptree." [Online]. Available: https://github.com/tox-dev/pipdeptree

[3] The Go Authors, "go list," Command go documentation. [Online]. Available: https://pkg.go.dev/cmd/go

[4] Apache Software Foundation, "dependency:tree," Apache Maven Dependency Plugin. [Online]. Available: https://maven.apache.org/plugins/maven-dependency-plugin/tree-mojo.html